ADAPTIVE DISTRIBUTED AI INFERENCE

IN HETEROGENEOUS EDGE NETWORKS

Three research directions · One problem space · From scheduling to orchestration

Paper 1

ASET

Adaptive Scheduling

Paper 2

Flocky

Decentralized Orchestration

Paper 3

DATOR

Dynamic Distributed Inference

Presented by Arjun A S · SCT23CS017

Dept. of Computer Science & Engineering · Sree Chitra Thirunal College of Engineering

Guide: Prof. Arun K

Why Does Edge AI Need Smarter Coordination?

Variable Compute & Load

Nodes: microcontrollers to GPUs
Hot-spots form; edge saturates

Dynamic Network Latency

Delay shifts across edge hops
Routes change continuously

Real-Time Inference Needs

Strict end-to-end latency targets
Node failures break pipelines

THE CENTRAL QUESTION

"Who processes this workload right now?"

As load, latency, and node state change — the answer changes too.

Core Concepts in Edge AI Coordination

Heterogeneity

Nodes vary: CPU, GPU, OS, memory
No uniform scheduling rule works

Scheduling

Select: model variant + resource cluster
Meet delay, accuracy, throughput

Orchestration

Coordinate deployment across nodes
Lifecycle and placement management

Adaptation

Revise decisions as conditions change
Workload, topology, availability

Three Papers · Three Lenses on One Problem

ASET

"Which scheduling policy should I use right now?"

Adaptive scheduling via DRL

Policy selection meta-scheduler

Flocky

"How can nodes coordinate without a central controller?"

Intent-based orchestration

Decentralized discovery + OAM

DATOR

"How can inference survive node and link failures?"

Self-organized multi-stage

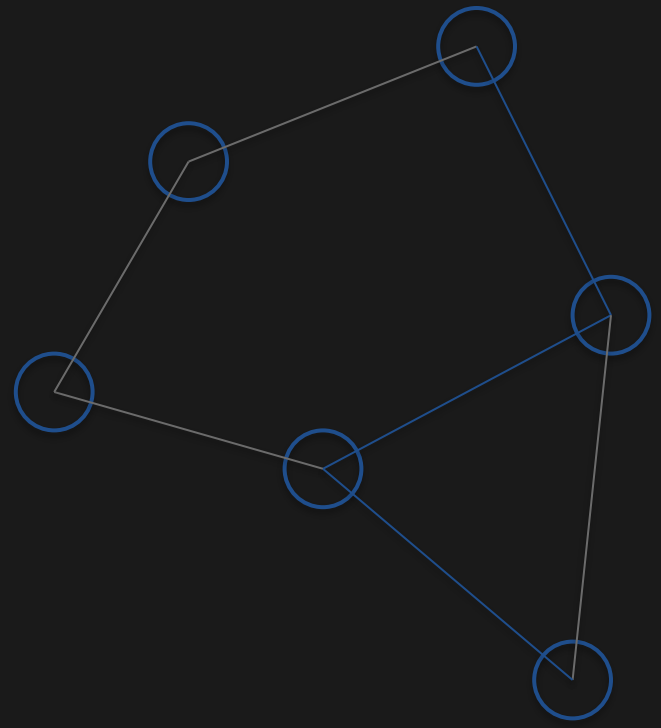
Dynamic rerouting + resilience

Scheduling → Orchestration → Resilient Inference

01

ASET

Adaptive Scheduling for Edge & Cloud Inference



Scheduling Inference Workloads in the Computing Continuum

Paper Metadata

Authors: G. Castellano, J-J. Nieto, A. Angi,
F. Á. Terribas, J. Luque, F. D. Andilla,
A. Sacco, F. Esposito, F. Risso
Venue: IEEE ICDCS Workshops 2025
Pages: 297–302
DOI: 10.1109/ICDCSW63273.2025.00056

ROLE IN SEMINAR

LAYER 1

Adaptive Decision Making

Scheduling DNN Inference Across the Continuum

Stream Requirements

Max end-to-end delay
Min accuracy (mAP)
Input / data rate
Input size

Model Variants

Accuracy · delay · throughput
CPU/GPU · batch size
Number of replicas
Deployment config

Resource Clusters

Edge / access network
Central-office resources
ISP / data-centre
Cloud resources

Select: feasible model variant + resource cluster → all constraints satisfied

Objective: maximise the fraction of successfully served inference queries

Why One Fixed Policy Is Insufficient

Closest
Minimise network distance / delay

Load Balance
Prefer resource with lowest current load

Farthest
Preserve nearby capacity; use distant nodes

Cheaper
Cost-oriented within available delay budget

Least Impedance
Minimise end-to-end latency

THE LIMITATION

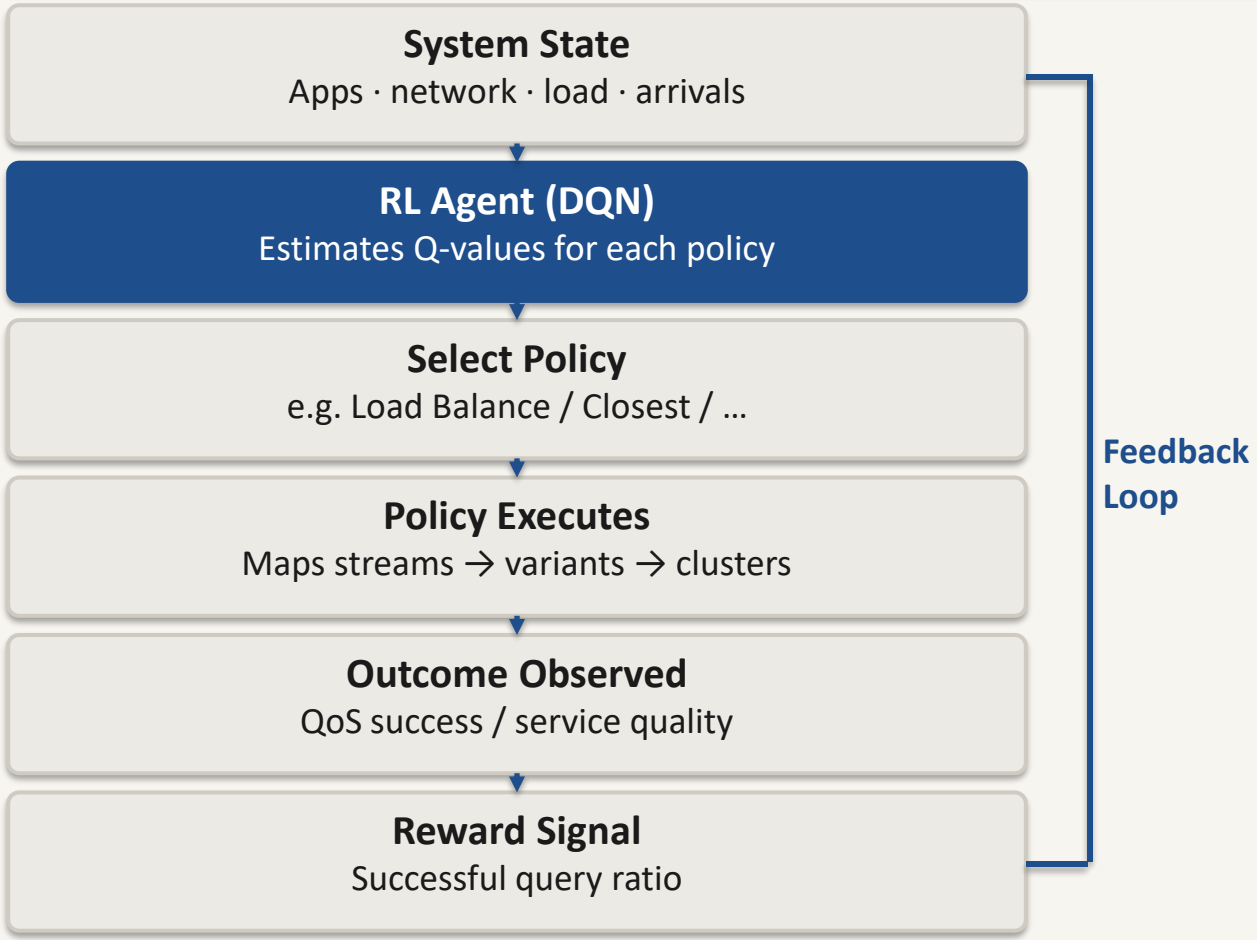
Low load →
nearby edge is ideal

High load →
same node overloads

Changing rates →
best policy shifts

No single policy stays optimal across all conditions.

ASET as a Meta-Scheduler



KEY DISTINCTION

ASET does NOT learn which node to use.

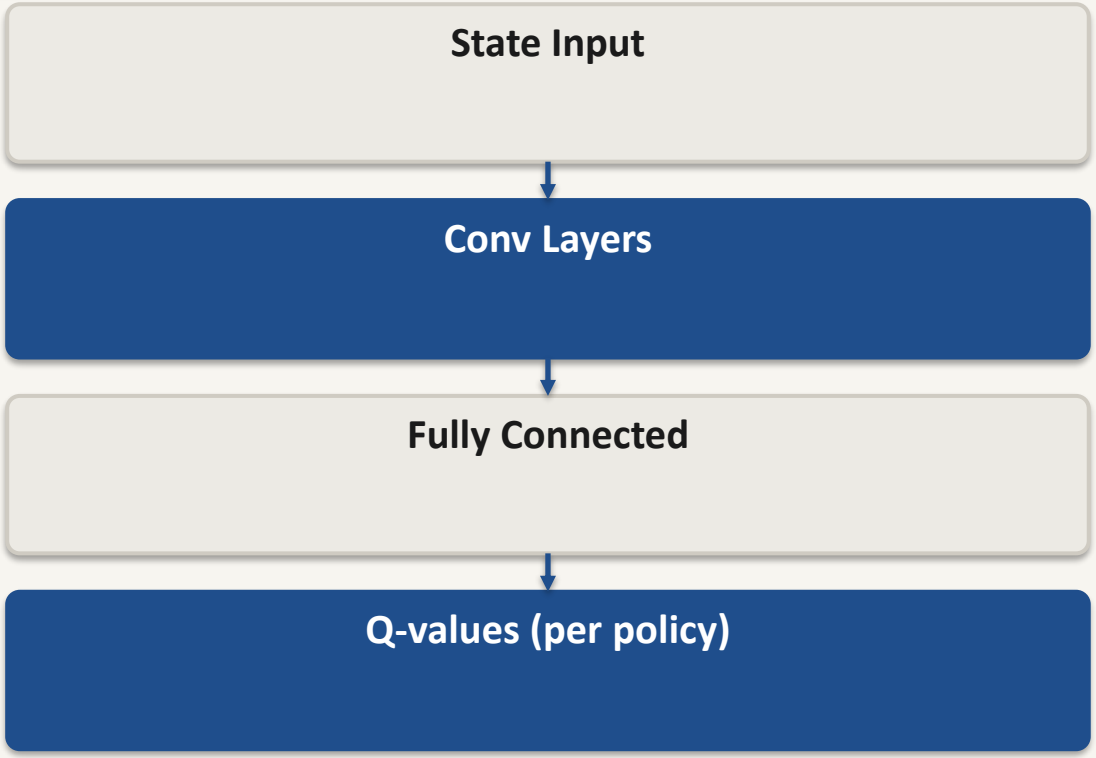
It learns WHICH POLICY to apply for the current state.

T = 25 s scheduling window

Trained: multiple topologies and workload configurations

MDP Formulation & DQN Architecture

State S_t	Apps · network topology · arrivals · load
Action A_t	Choice of scheduling policy
Reward	Successful query servicing quality
Transition	Next state after policy applied for T=25s



Q(state, policy) = expected return for selecting that policy now

ε-greedy exploration · dynamic epsilon update

Results Across Topologies & Workloads

dc-cloud

ASET not uniformly best
Performance is topology-dependent
Can underperform some statics in higher load

co-dc-cloud

Improves over evaluated baselines
Holds even outside training arrival rate
Moderate generalization gains

full-edge

Strongest improvement
↑ successful-query ratio
Generalises to dynamic workloads

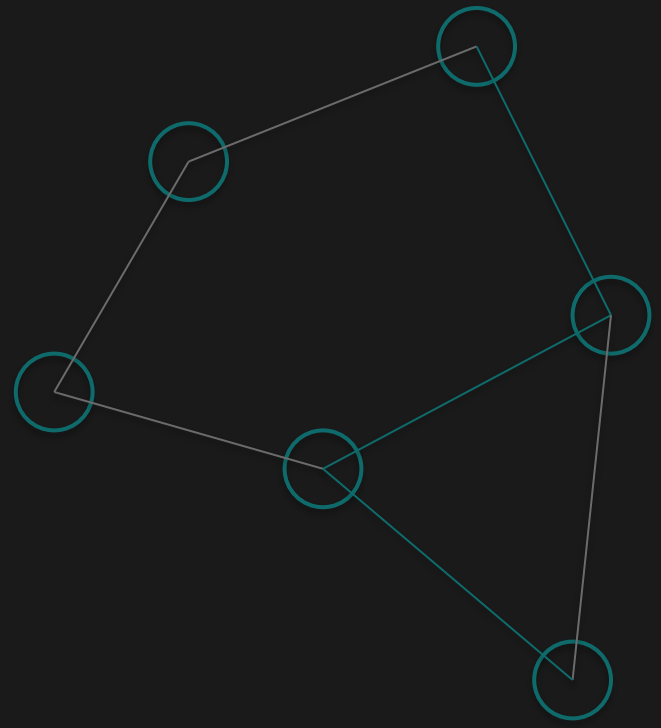
Key Takeaway

State-aware policy selection adapts across conditions.
Performance gains are topology-specific — not universal.

02

Flocky

Decentralized Intent-based Edge Orchestration



Decentralized Intent-based Edge Orchestration Using OAM

Paper Metadata

Authors: T. Goethals, M. Sebrechts, M. Al-Naday,
F. De Turck, B. Volckaert
Venue: IEEE Trans. Services Computing
Volume 19, Issue 1, pp. 380–395, 2026
DOI: 10.1109/TSC.2025.3631387

ROLE IN SEMINAR

LAYER 2

Decentralized Coordination

Cloud Assumptions Don't Hold at the Edge

Cloud Assumptions

Small set of powerful machines
Centralized orchestrator (Kubernetes)
High-capacity, stable infrastructure
Fast, reliable networking

VS

Edge Reality

Many heterogeneous, limited nodes
No single entity has global view
Changing topology and connectivity
Resource-constrained devices

Can orchestration itself become decentralized?

From Workload Intent to Placement Decision

Workload Intent
Requirements expressed declaratively

↓

Open Application Model
Structured component description + edge traits

↓

Node Metadata
Per-node capability advertisement

↓

Capability Matching
Requirements ↔ metadata per candidate node

↓

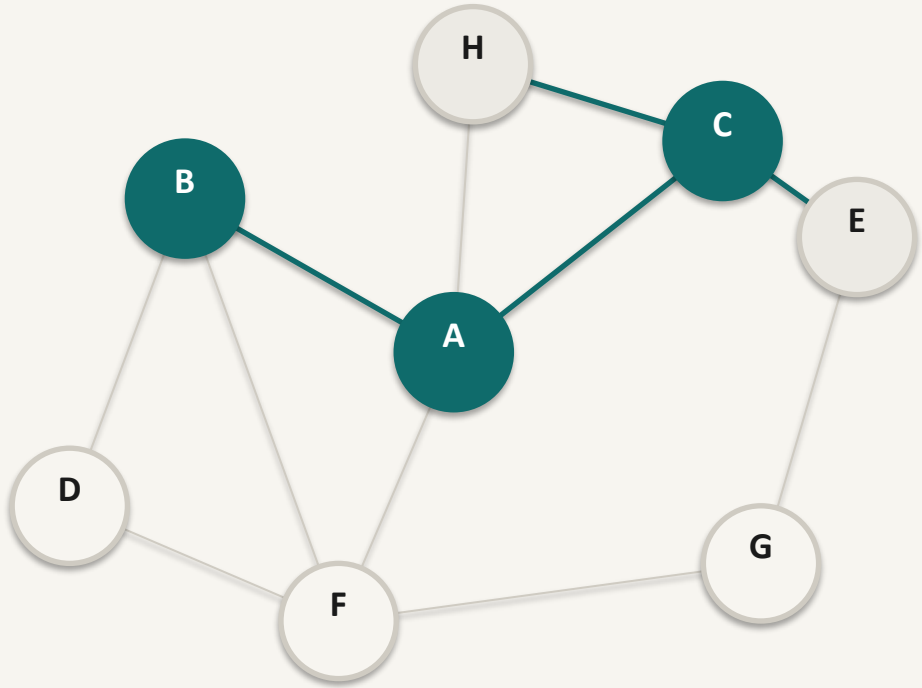
Placement Decision
Best-matching node(s) selected & deployed

OAM EDGE TRAITS

Orchestration considers more than raw CPU:

- ✦ Network encryption
- ✦ Attestation
- ✦ Green energy
- ✦ Secure runtime
- ✦ Distance constraints

Building Local Topology Knowledge



■ Teal = selected node + local links ■ Grey = distant / unvisited

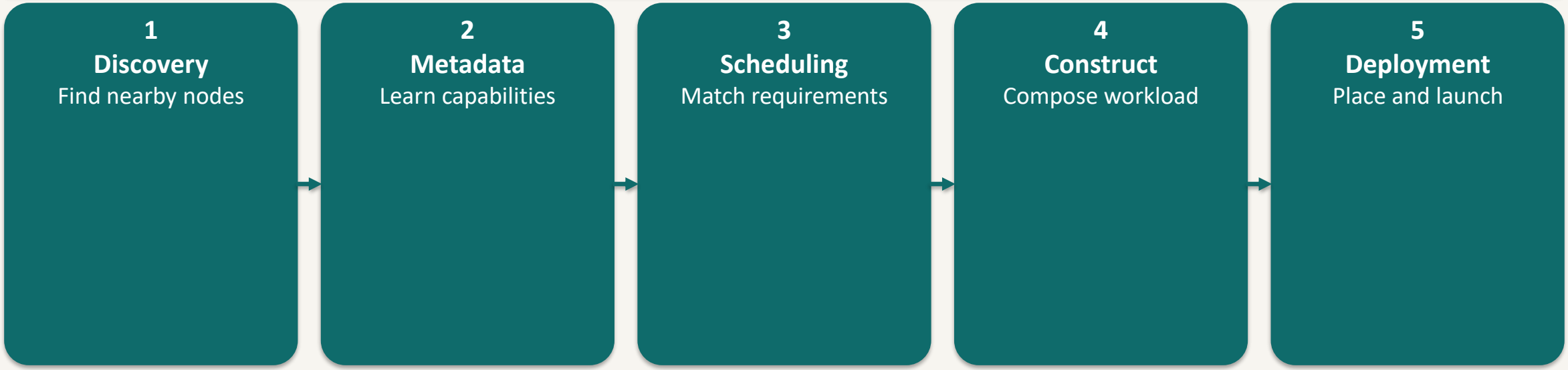
As reported by the paper

DISCOVERY STEPS

1. Local node broadcasts capability info
2. Nearby nodes respond with metadata
3. PING measures round-trip latency
4. Metadata cached for placement decisions
5. Local topology view built — no global state

>97% of neighbours discovered in 2 rounds

Five-Stage Decentralized Orchestration



Orchestration decisions are distributed across participating nodes.

No single global scheduler — each node participates in placement collectively.

Key Findings from the Paper

>97%

of viable neighbours discovered
in two discovery rounds
as reported by the paper

84%

less memory than centralized
Kubernetes (reported evaluation)
as reported by the paper

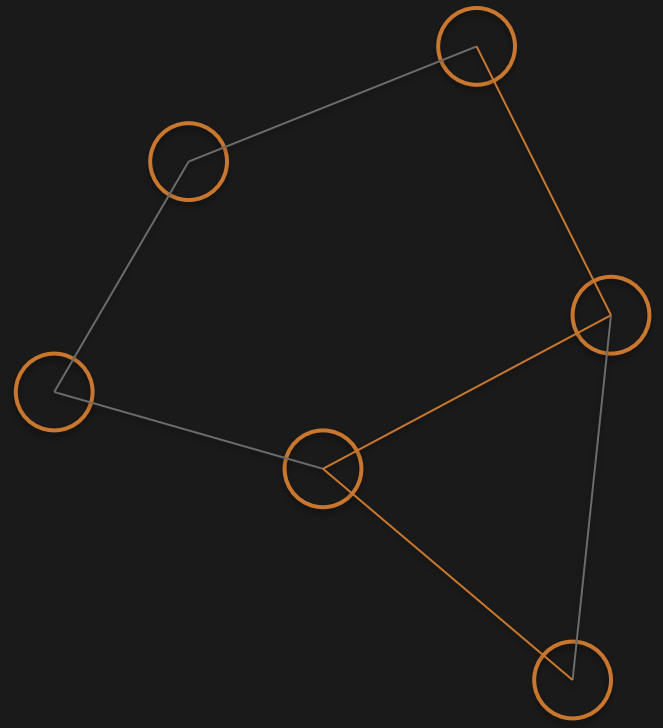
Architectural Contribution

Flocky demonstrates decentralized edge orchestration without a global controller.
Intent + node metadata + local discovery enables informed placement at scale.

03

DATOR

Self-Organized Neural Network Inference in Dynamic Edges



Self-Organized Neural Network Inference in Dynamic Edge Networks

Paper Metadata

Authors: M. Schrauth, M. Thome,
T. Ohlenforst, F. Kreyß
Venue: Applied Sciences
Volume 15, Issue 23, Article 12615, 2025
DOI: 10.3390/app152312615

ROLE IN SEMINAR

LAYER 3

Dynamic Execution & Resilience

Static Placement Cannot Survive Dynamic Edges

Node Dropout

Nodes disconnect unexpectedly
Inference pipeline breaks

Intermittent Connectivity

Links flicker; packets lost
Transmission times spike

Heterogeneous Hardware

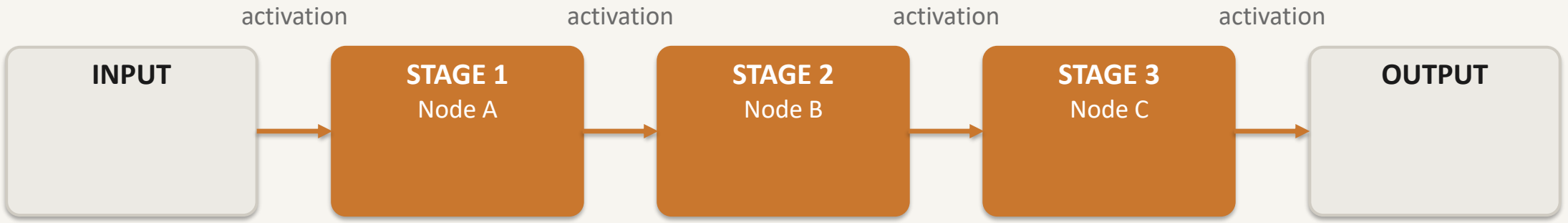
Nodes vary in compute power
One model does not fit all

Changing Workload

Inference demand shifts
Fixed allocation under/over-serves

Design Goal: maintain distributed inference without a central controller or single point of failure.

Splitting the Neural Network Across Edge Nodes



How It Works

The DNN is partitioned into layer groups (stages).
Each stage maps to a suitable edge node.
Intermediate activations flow between nodes.
If a node fails — the system finds an alternate path.

Work Assignment Without Central Control

New Work Package
Inference stage needs assignment

↓

Broadcast Request
Current node announces task to neighbours

↓

Eligible Nodes Respond
Nodes with capacity + module reply

↓

Node Accepts
Best candidate dispatched

↓

Stage Executes
Result forwarded to next stage

NODE DECISION CRITERIA

Module present?
Does node hold the needed layer group?

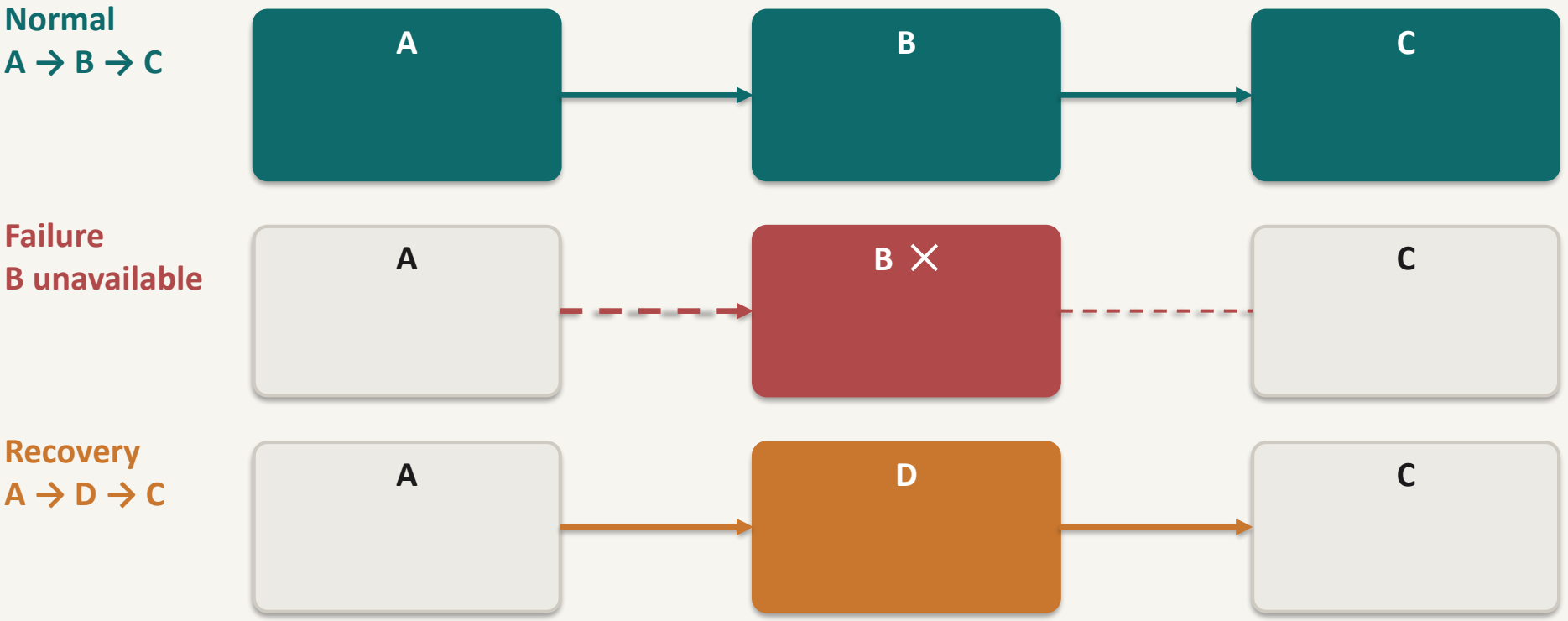
Utilization OK?
Load within acceptance threshold?

Hardware capable?
Can execute stage efficiently?

Weighted-auction factors:

Capability · Latency · Energy · Utilization
(Contract Net Protocol concepts)

Inference Continues Despite Node Failures



Fault handling is integrated into the inference path — no centralized recovery controller.

Performance Across Network Conditions

Stable

No node failures
Baseline performance

Unstable

Intermittent node failures
System continues via rerouting

Impaired Comm.

5% message loss
Reduced throughput, stable ops

Severe Comm.

~10× longer transmissions
System approaches saturation

General Performance Pattern

Low load	Latency remains relatively low
Increasing load	More nodes join distributed execution
Near saturation	Throughput & latency approach capacity

Designed to maintain operation even when not all nodes and links are permanently available.

Main Question & Method

	ASET	Flocky	DATOR
Main Question	Which scheduling policy for the current state?	How can orchestration be decentralized?	How can inference survive dynamic nodes & links?
Method	Deep Q-Network Policy selection via DRL	Intent + OAM + Metadata Decentralized discovery	Multi-stage distributed Local negotiation + rerouting
Decision Level	Scheduling policy selection	App/component placement	Inference-stage assignment & routing

Strengths & Challenges

	ASET	Flocky	DATOR
Key Strength	Workload / network adaptation	Decentralized coordination & local knowledge	Self-organization & resilience
Challenge Addressed	Changing system conditions	Scalable decentralized orchestration	Node / link failures & dynamic environments
Not Competing	Adaptive decision making (layer 1)	Decentralized coordination (layer 2)	Dynamic execution & recovery (layer 3)

Three Papers · One Problem Space

ASET

Adaptive Scheduling

State-aware policy selection via DRL

Flocky

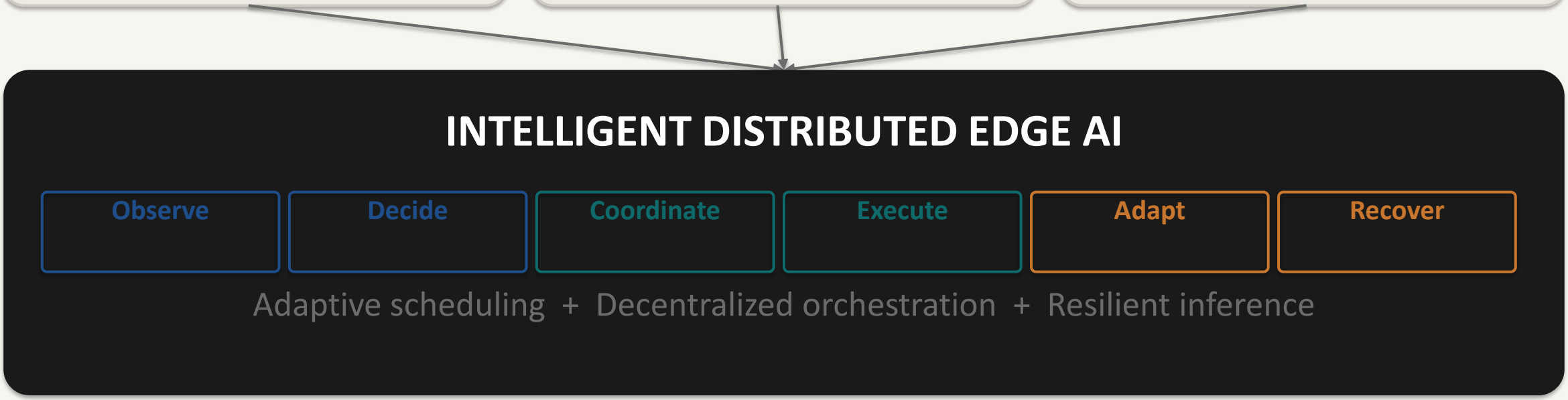
Decentralized Orchestration

Intent + OAM + local discovery

DATOR

Dynamic Resilient Inference

Multi-stage + rerouting + self-org



Open Questions in Edge AI Coordination

Demonstrated by the Three Papers

Adaptive scheduling (ASET)
Decentralized orchestration (Flocky)
Dynamic self-organization (DATOR)

- Richer state-aware scheduling
- Decentralized discovery at scale
- Energy-aware dynamic routing
- Multimodal AI inference at edge
- Cross-layer integration
- Fault tolerance in large clusters

Integration Vision

A unified edge system that can simultaneously: observe state, select resources, coordinate nodes, adapt, and recover.

Key Takeaways & References

ASET

Deep Q-Learning meta-scheduler
Selects policy, not node

Flocky

Decentralized OAM orchestration
Local discovery and matching

DATOR

Self-organized neural inference
Dynamic rerouting resilience

Adaptive Scheduling + Decentralized Orchestration + Resilient Inference = Intelligent Edge AI

References

[1] G. Castellano et al., "Scheduling Inference Workloads in the Computing Continuum with RL," IEEE ICDCS Wkshps, 2025, pp. 297-302. DOI: 10.1109/ICDCSW63273.2025.00056

[2] T. Goethals et al., "Flocky: Decentralized Intent-based Edge Orchestration Using OAM," IEEE TSC, vol. 19, no. 1, pp. 380-395, 2026. DOI: 10.1109/TSC.2025.3631387

[3] M. Schrauth et al., "Self-Organized Neural Network Inference in Dynamic Edge Networks," Applied Sciences, vol. 15, no. 23, 12615, 2025. DOI: 10.3390/app152312615